

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

2265 · Count Nodes Equal to Average of Subtree

LeetCode · Daily Question

Medium

Tree

DFS / Post-order

1. What Type of Question Is This?

This is a classic **post-order tree DP** problem. To know anything about a node's subtree — its total sum, its total node count — you first need that same information from both of its children. That dependency direction (children before parent) is exactly what post-order traversal gives you for free. Whenever a problem asks you to compute something about "this node's subtree" that depends on aggregating over all its descendants, the standard tool is a **single post-order DFS that returns aggregated values up the call stack**, rather than re-scanning each subtree separately.

2. What Is Given?

- The **root** of a binary tree.
- A **subtree** of a node is that node together with all of its descendants.
- The **average** of a subtree with n nodes is the sum of all n node values divided by n , **rounded down** to the nearest integer (integer/floor division).

3. What Do We Need to Find?

The number of nodes in the tree where the node's own value **equals** the (floored) average of the values in its own subtree.

Function Signature

■ C++

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 * };
 */
class Solution {
public:
    int averageOfSubtree(TreeNode* root) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  root = [4,8,5,0,1,null,6]
Output: 5
```

Tree: 4 is the root, with left child 8 (which has children 0 and 1) and right child 5 (which has a right child 6, no left child). Checking every node's subtree average: node 4 $\rightarrow (4+8+5+0+1+6)/6 = 24/6 = 4$ ✓. Node 5 $\rightarrow (5+6)/2 = 11/2 = 5$ ✓. Node 0 $\rightarrow 0/1 = 0$ ✓. Node 1 $\rightarrow 1/1 = 1$ ✓. Node 6 $\rightarrow 6/1 = 6$ ✓. Node 8 $\rightarrow (8+0+1)/3 = 9/3 = 3$, which does **not** equal 8, so it doesn't count. That's 5 matching nodes $\rightarrow 5$.

Example 2 — single node

```
Input:  root = [1]
Output: 1
```

A single-node tree's only subtree is itself: average = $1/1 = 1$, which equals the node's own value $\rightarrow 1$.

Constraints

- The number of nodes in the tree is in the range $[1, 1000]$.
- $0 \leq \text{Node.val} \leq 1000$

Up to 1000 nodes, so anything up to roughly $O(n^2)$ will run comfortably — but the clean $O(n)$ single-pass solution is the one worth internalizing, since it's the standard shape for this entire family of tree-aggregation problems.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Identify what each node needs to know about itself.** To check whether a node qualifies, you need exactly two numbers about its subtree: the **sum** of all values in it, and the **count** of nodes in it. Everything else (the average, the comparison) follows immediately from those two numbers.
- **Step 2 — Notice that a node's (sum, count) is built from its children's (sum, count).** $\text{subtreeSum}(\text{node}) = \text{node.val} + \text{subtreeSum}(\text{left}) + \text{subtreeSum}(\text{right})$, and $\text{subtreeCount}(\text{node}) = 1 + \text{subtreeCount}(\text{left}) + \text{subtreeCount}(\text{right})$. Both children must be fully processed before the parent can compute its own values — this is precisely what **post-order** traversal (left, right, then the node itself) guarantees.
- **Step 3 — Avoid recomputing the same subtree information over and over.** If you separately re-scan a node's entire subtree every time you want its sum and count, you end up doing that work once per node in the tree, and each scan itself touches every descendant — a lot of duplicated effort for nodes deep in the tree.
- **Step 4 — Let each recursive call return what its caller needs.** Instead of recomputing, have the DFS function itself *return* the (sum, count) pair for the subtree it just processed. The parent simply combines its own value with what its two children already handed back — no re-scanning required.
- **Step 5 — Check the condition on the way back up.** The moment a node's (sum, count) is known, immediately check whether $\text{sum} \div \text{count} == \text{node.val}$ (integer division) and update a running counter if so, all within the same traversal that's already computing the aggregates.

Which Algorithm Is Best?

A **single post-order DFS** that returns the (sum, count) pair for each subtree and checks the average condition immediately upon returning from a node's children is the natural optimal solution: $O(n)$ time, since every node is visited exactly once, and $O(h)$ extra space for the recursion stack (h = tree height). The brute-force and "better" tiers below build up to this by first showing the cost of re-scanning subtrees separately, then an intermediate version that avoids re-scanning but still uses more bookkeeping (a hash map) than necessary.

Approach 1 — Brute Force (Re-scan Each Subtree Separately)

Idea: For every node in the tree, run a completely separate traversal of just that node's subtree to compute its sum and count from scratch, check the average condition, and only then move on to check the next node. The two traversals (one to visit every node, one to re-measure each subtree) are kept fully independent.

Steps

- Write a helper `sumAndCount(node)` that recursively computes the full (sum, count) of node's subtree by descending into both children.
- Write a second traversal, `visitEveryNode(node)`, that visits every node in the tree.
- At each visited node, call `sumAndCount(node)` fresh — this re-walks the entire subtree rooted there.
- Compare `sum ÷ count` to `node.val` (integer division); increment a counter on a match.
- Recurse `visitEveryNode` into both children to check every other node the same way.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};

class Solution {
public:
    // Re-scans an entire subtree from scratch just to get its (sum, count).
    pair<int, int> sumAndCount(TreeNode* node) {
        if (!node) return {0, 0};
        auto l = sumAndCount(node->left);
        auto r = sumAndCount(node->right);
        return {node->val + l.first + r.first, 1 + l.second + r.second};
    }

    int count = 0;

    void visitEveryNode(TreeNode* node) {
        if (!node) return;
        auto [sum, cnt] = sumAndCount(node); // full re-scan of this node's subtree
        if (sum / cnt == node->val) count++;
        visitEveryNode(node->left);
        visitEveryNode(node->right);
    }

    int averageOfSubtree(TreeNode* root) {
        count = 0;
        visitEveryNode(root);
        return count;
    }
};

int main() {
    TreeNode* n0 = new TreeNode(0);
    TreeNode* n1 = new TreeNode(1);
    TreeNode* n6 = new TreeNode(6);
    TreeNode* n8 = new TreeNode(8, n0, n1);
```

```

TreeNode* n5 = new TreeNode(5, nullptr, n6);
TreeNode* root = new TreeNode(4, n8, n5);

Solution sol;
cout << sol.averageOfSubtree(root) << endl;    // Expected output: 5
return 0;
}

```

■ Python

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # Re-scans an entire subtree from scratch just to get its (sum, count).
    def sumAndCount(self, node):
        if node is None:
            return (0, 0)
        ls, lc = self.sumAndCount(node.left)
        rs, rc = self.sumAndCount(node.right)
        return (node.val + ls + rs, 1 + lc + rc)

    def averageOfSubtree(self, root):
        self.count = 0

        def visitEveryNode(node):
            if node is None:
                return
            total, cnt = self.sumAndCount(node)    # full re-scan of this node's subtree
            if total // cnt == node.val:
                self.count += 1
            visitEveryNode(node.left)
            visitEveryNode(node.right)

        visitEveryNode(root)
        return self.count

if __name__ == "__main__":
    n0, n1, n6 = TreeNode(0), TreeNode(1), TreeNode(6)
    n8 = TreeNode(8, n0, n1)
    n5 = TreeNode(5, None, n6)
    root = TreeNode(4, n8, n5)

    print(Solution().averageOfSubtree(root))    # Expected output: 5

```

■ JavaScript

```

class TreeNode {
    constructor(val, left = null, right = null) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    // Re-scans an entire subtree from scratch just to get its [sum, count].
    sumAndCount(node) {
        if (!node) return [0, 0];
        const [ls, lc] = this.sumAndCount(node.left);
        const [rs, rc] = this.sumAndCount(node.right);
        return [node.val + ls + rs, 1 + lc + rc];
    }

    averageOfSubtree(root) {
        let count = 0;

```

```

const visitEveryNode = (node) => {
  if (!node) return;
  const [sum, cnt] = this.sumAndCount(node);    // full re-scan of this node's subtree
  if (Math.floor(sum / cnt) === node.val) count++;
  visitEveryNode(node.left);
  visitEveryNode(node.right);
};

visitEveryNode(root);
return count;
}
}

const n0 = new TreeNode(0), n1 = new TreeNode(1), n6 = new TreeNode(6);
const n8 = new TreeNode(8, n0, n1);
const n5 = new TreeNode(5, null, n6);
const root = new TreeNode(4, n8, n5);

console.log(new Solution().averageOfSubtree(root));    // Expected output: 5

```

Complexity: `sumAndCount(node)` costs $O(\text{size of node's subtree})$. Summed across every node in the tree, the total work is $O(n^2)$ in the worst case (e.g. a skewed tree, where the root's subtree alone is re-scanned in $O(n)$, the next node's in $O(n-1)$, and so on). Comfortably fast for $n \leq 1000$, but each subtree's information is being computed from scratch far more often than necessary.

Approach 2 — Better (Compute Once into a Map, Then Check in a Second Pass)

Idea: Stop re-scanning subtrees. Do one post-order pass that computes every node's (sum, count) exactly once and stores it in a hash map keyed by node, then do a separate pass that simply looks up each node's precomputed values to check the average condition.

Steps

- Run one post-order DFS (computeAndStore) that visits every node exactly once, computing (sum, count) from the children's already-computed values.
- Store each node's (sum, count) in a hash map as soon as it's computed.
- Run a second traversal (checkAll) over the same tree, looking up each node's (sum, count) from the map in $O(1)$ instead of recomputing it.
- At each node, compare $\text{sum} \div \text{count}$ to node.val and increment the counter on a match.
- Return the counter after the second traversal finishes.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};

class Solution {
public:
    unordered_map<TreeNode*, pair<int,int>> memo;    // node -> (subtree sum, subtree count)

    pair<int,int> computeAndStore(TreeNode* node) {
        if (!node) return {0, 0};
        auto l = computeAndStore(node->left);
        auto r = computeAndStore(node->right);
        pair<int,int> result = {node->val + l.first + r.first, 1 + l.second + r.second};
        memo[node] = result;                        // store it for the second pass to reuse
        return result;
    }

    int averageOfSubtree(TreeNode* root) {
        memo.clear();
        computeAndStore(root);                     // pass 1: fill the map

        int count = 0;
        function<void(TreeNode*)> checkAll = [&](TreeNode* node) {
            if (!node) return;
            auto [sum, cnt] = memo[node];           // O(1) lookup instead of recomputing
            if (sum / cnt == node->val) count++;
            checkAll(node->left);
            checkAll(node->right);
        };
        checkAll(root);                             // pass 2: check every node

        return count;
    }
};
```



```

int main() {
    TreeNode* n0 = new TreeNode(0);
    TreeNode* n1 = new TreeNode(1);
    TreeNode* n6 = new TreeNode(6);
    TreeNode* n8 = new TreeNode(8, n0, n1);
    TreeNode* n5 = new TreeNode(5, nullptr, n6);
    TreeNode* root = new TreeNode(4, n8, n5);

    Solution sol;
    cout << sol.averageOfSubtree(root) << endl;    // Expected output: 5
    return 0;
}

```

Python

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def averageOfSubtree(self, root):
        memo = {}    # node -> (subtree sum, subtree count)

        def computeAndStore(node):
            if node is None:
                return (0, 0)
            ls, lc = computeAndStore(node.left)
            rs, rc = computeAndStore(node.right)
            result = (node.val + ls + rs, 1 + lc + rc)
            memo[node] = result    # store it for the second pass to reuse
            return result

        computeAndStore(root)    # pass 1: fill the map

        count = 0

        def checkAll(node):
            nonlocal count
            if node is None:
                return
            total, cnt = memo[node]    # O(1) lookup instead of recomputing
            if total // cnt == node.val:
                count += 1
            checkAll(node.left)
            checkAll(node.right)

        checkAll(root)    # pass 2: check every node
        return count

if __name__ == "__main__":
    n0, n1, n6 = TreeNode(0), TreeNode(1), TreeNode(6)
    n8 = TreeNode(8, n0, n1)
    n5 = TreeNode(5, None, n6)
    root = TreeNode(4, n8, n5)

    print(Solution().averageOfSubtree(root))    # Expected output: 5

```

JavaScript

```

class TreeNode {
    constructor(val, left = null, right = null) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {

```

```

averageOfSubtree(root) {
  const memo = new Map(); // node -> [subtree sum, subtree count]

  const computeAndStore = (node) => {
    if (!node) return [0, 0];
    const [ls, lc] = computeAndStore(node.left);
    const [rs, rc] = computeAndStore(node.right);
    const result = [node.val + ls + rs, 1 + lc + rc];
    memo.set(node, result); // store it for the second pass to reuse
    return result;
  };

  computeAndStore(root); // pass 1: fill the map

  let count = 0;
  const checkAll = (node) => {
    if (!node) return;
    const [sum, cnt] = memo.get(node); // O(1) lookup instead of recomputing
    if (Math.floor(sum / cnt) === node.val) count++;
    checkAll(node.left);
    checkAll(node.right);
  };
  checkAll(root); // pass 2: check every node

  return count;
}

const n0 = new TreeNode(0), n1 = new TreeNode(1), n6 = new TreeNode(6);
const n8 = new TreeNode(8, n0, n1);
const n5 = new TreeNode(5, null, n6);
const root = new TreeNode(4, n8, n5);

console.log(new Solution().averageOfSubtree(root)); // Expected output: 5

```

Complexity: Each node's (sum, count) is computed exactly once — $O(n)$ time for the first pass, plus $O(n)$ for the second pass of map lookups, so $O(n)$ overall. However, it needs $O(n)$ extra space for the hash map (on top of the $O(h)$ recursion stack), and does the work in two separate traversals when one would do.

Approach 3 — Optimal (One Post-order Pass, Check on the Way Back Up)

Idea: Merge the two separate passes of Approach 2 into one. There's no need to store every node's (sum, count) in a map at all — each node only ever needs its own (sum, count) once, right when its children's recursive calls return. Check the average condition at that exact moment, then simply return the (sum, count) pair up to the parent, who does the same thing.

Steps

- Write a single recursive function `dfs(node)` that returns the pair (sum, count) for node's subtree.
- Base case: if node is null, return (0, 0).
- Recursively call `dfs` on the left and right children first (post-order: children before the node itself).
- Combine: `sum = node.val + leftSum + rightSum`, `count = 1 + leftCount + rightCount`.
- Immediately check `sum ÷ count == node.val` and increment a class-level counter if it matches — no map, no second traversal.
- Return (sum, count) so the parent call can use it the same way; the top-level call is `dfs(root)`, and the final counter value is the answer.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};

class Solution {
public:
    int count = 0;

    // Returns {sum, count} for node's subtree, checking the average condition on the way back up.
    pair<int,int> dfs(TreeNode* node) {
        if (!node) return {0, 0};

        auto l = dfs(node->left);
        auto r = dfs(node->right);

        int sum = node->val + l.first + r.first;
        int cnt = 1 + l.second + r.second;

        if (sum / cnt == node->val) count++; // check immediately, no extra storage needed

        return {sum, cnt};
    }

    int averageOfSubtree(TreeNode* root) {
        count = 0;
        dfs(root);
        return count;
    }
};

int main() {
```

```

TreeNode* n0 = new TreeNode(0);
TreeNode* n1 = new TreeNode(1);
TreeNode* n6 = new TreeNode(6);
TreeNode* n8 = new TreeNode(8, n0, n1);
TreeNode* n5 = new TreeNode(5, nullptr, n6);
TreeNode* root = new TreeNode(4, n8, n5);

Solution sol;
cout << sol.averageOfSubtree(root) << endl;    // Expected output: 5
return 0;
}

```

■ Python

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def averageOfSubtree(self, root):
        self.count = 0

        # Returns (sum, count) for node's subtree, checking the average condition
        # on the way back up.
        def dfs(node):
            if node is None:
                return (0, 0)

            ls, lc = dfs(node.left)
            rs, rc = dfs(node.right)

            total = node.val + ls + rs
            cnt = 1 + lc + rc

            if total // cnt == node.val:      # check immediately, no extra storage needed
                self.count += 1

            return (total, cnt)

        dfs(root)
        return self.count

if __name__ == "__main__":
    n0, n1, n6 = TreeNode(0), TreeNode(1), TreeNode(6)
    n8 = TreeNode(8, n0, n1)
    n5 = TreeNode(5, None, n6)
    root = TreeNode(4, n8, n5)

    print(Solution().averageOfSubtree(root))    # Expected output: 5

```

■ JavaScript

```

class TreeNode {
    constructor(val, left = null, right = null) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    averageOfSubtree(root) {
        let count = 0;

        // Returns [sum, count] for node's subtree, checking the average condition
        // on the way back up.
        const dfs = (node) => {
            if (!node) return [0, 0];

```

```

    const [ls, lc] = dfs(node.left);
    const [rs, rc] = dfs(node.right);

    const sum = node.val + ls + rs;
    const cnt = 1 + lc + rc;

    if (Math.floor(sum / cnt) === node.val) count++; // check immediately, no extra storage

    return [sum, cnt];
};

dfs(root);
return count;
}
}

const n0 = new TreeNode(0), n1 = new TreeNode(1), n6 = new TreeNode(6);
const n8 = new TreeNode(8, n0, n1);
const n5 = new TreeNode(5, null, n6);
const root = new TreeNode(4, n8, n5);

console.log(new Solution().averageOfSubtree(root)); // Expected output: 5

```

Complexity: A single $O(n)$ traversal — every node is visited exactly once, and all work at each node (the additions and the comparison) is $O(1)$. Space is $O(h)$ for the recursion stack (h = tree height, up to $O(n)$ for a completely skewed tree, but typically $O(\log n)$ for a balanced one) — no auxiliary map needed at all, unlike Approach 2.

Side-by-Side Comparison

1. Brute Force	Re-scan each node's subtree from scratch to get its sum/count	$O(n^2)$ worst case	$O(h)$ recursion stack
2. Better	One pass fills a map with every node's (sum, count); a second pass reads it	$O(n)$	$O(n)$ map + $O(h)$ stack
3. Optimal	One post-order pass; check the condition the moment (sum, count) is known	$O(n)$	$O(h)$, no map

Key Takeaways to Remember

- **"Something about this node's subtree"** is the signature phrase for a post-order DFS that returns aggregated values (sums, counts, heights, etc.) up the call stack — children must finish before the parent can compute its own answer.
- Let the **return value** of the recursive call carry exactly what the parent needs — this avoids both re-scanning subtrees (Approach 1's mistake) and needing an auxiliary map to shuttle information between two separate passes (Approach 2's in-between step).
- Check any per-node condition **immediately when its required data becomes available** (right as a recursive call returns), rather than deferring it to a second traversal — it's free to do it inline and it removes a whole pass over the tree.
- Watch for integer/floor division specifically — "average rounded down" means $\text{sum} \div \text{count}$ using integer division, not float division followed by rounding.
- All three tiers were validated against **Example 1** (root = [4,8,5,0,1,null,6] → 5) across C++, Python, and JavaScript, and checked against Example 2 (root = [1] → 1) as well.